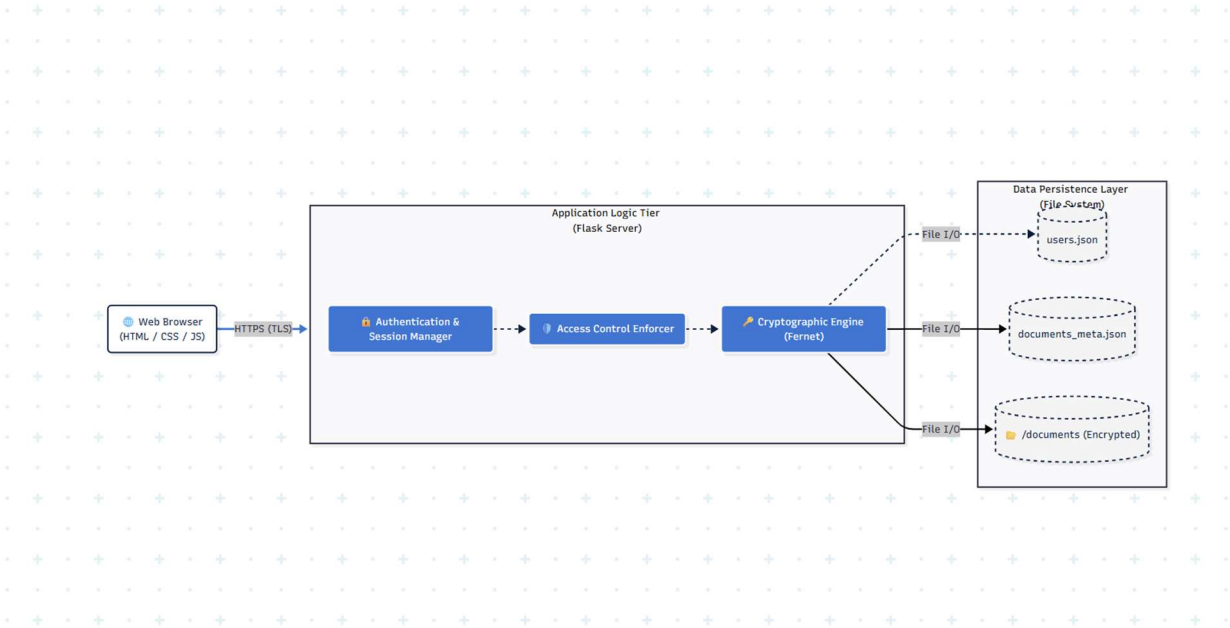
Security Design Document: Secure Document Sharing System

A. Architecture Overview

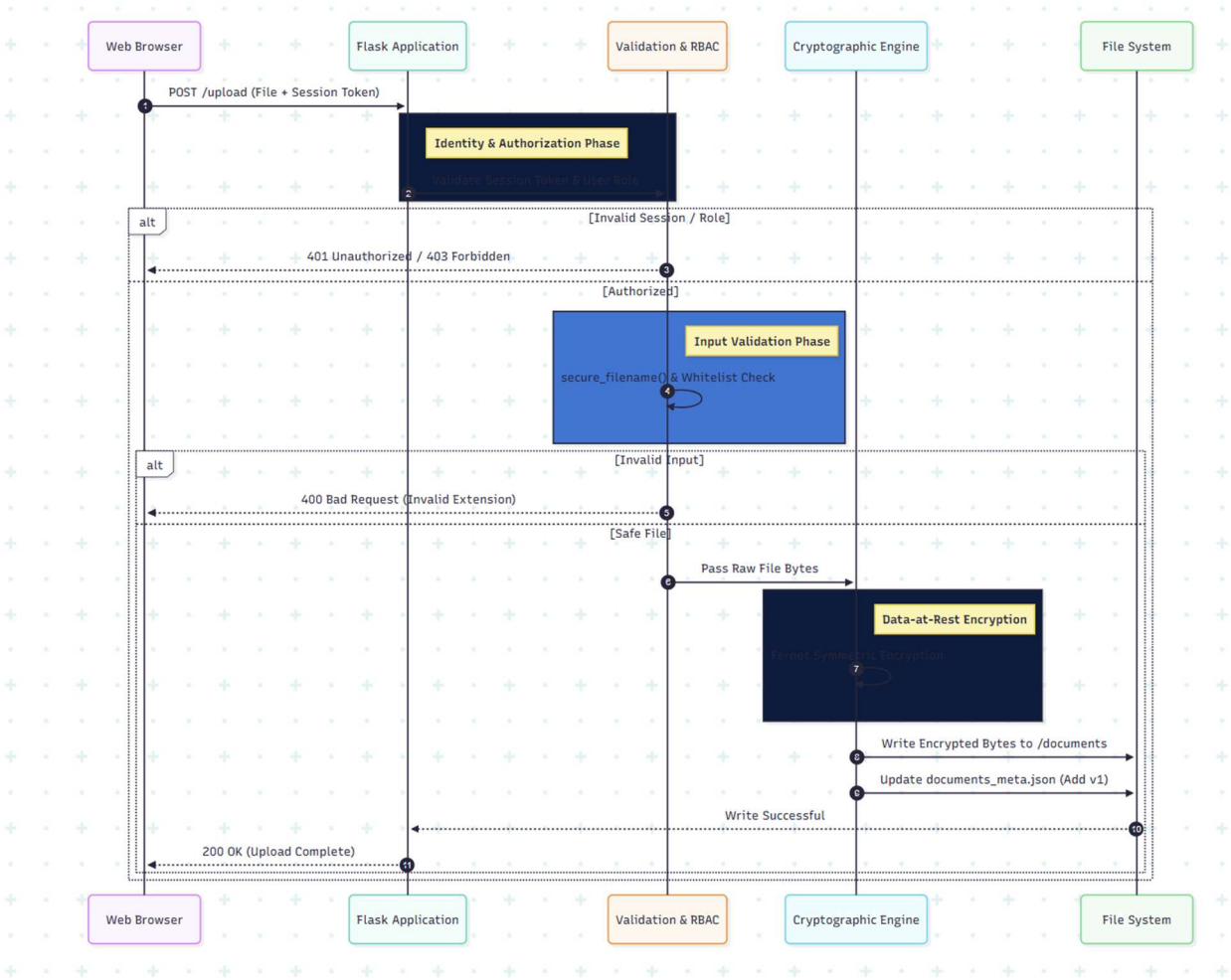
1. System Architecture Diagram



The Secure Document Sharing System utilizes a streamlined, lightweight three-tier architectural model. The presentation tier consists of a web-based client interface rendered via HTML, CSS, and JavaScript. This client communicates securely over HTTPS with the application logic tier, which is powered by a Python Flask server. The Flask application acts as the central orchestrator, processing all incoming HTTP requests, enforcing strict Role-Based Access Control (RBAC), and executing cryptographic operations. The final tier is the data persistence layer. Unlike

traditional web applications that rely on relational database management systems (RDBMS) like MySQL or PostgreSQL, this architecture intentionally relies on structured, local file-based storage using JSON files and raw binary storage for encrypted documents.

2. Data Flow Diagrams



The primary data flow for document upload and retrieval is heavily guarded by sequential security checkpoints. When a user attempts to upload a file, the request first passes through the session validation interceptor. Once identity is confirmed, the application logic extracts the file and performs input validation against a strict whitelist of allowed extensions. Upon validation, the file is passed to the cryptographic engine. The raw file bytes are symmetrically encrypted in

memory before any disk-write operations occur. Finally, the encrypted binary blob is saved to the designated storage directory, and the document's metadata (including ownership and version history) is updated in the documents_meta.json file.

3. Component Descriptions

The system is modularized into several core operational components:

- **Authentication & Session Manager:** This component is responsible for verifying user credentials against stored bcrypt hashes. Upon successful authentication, it utilizes the secrets library to generate a cryptographically secure 32-byte URL-safe session token. It tracks the user's IP address and user-agent, enforcing a strict 30-minute inactivity timeout to mitigate session hijacking risks.
- **Access Control Enforcer:** Implemented via custom Python decorators (@require_auth and @require_role), this component wraps protected routes. It evaluates the current user's session data against the global permission matrix to ensure that users can only interact with resources they are explicitly authorized to access, preventing vertical privilege escalation.
- **Cryptographic Engine:** This module isolates all encryption operations. It leverages the cryptography.fernet library to manage the master symmetric key. It guarantees that all documents written to the file system are mathematically unreadable without the corresponding secret key, protecting data-at-rest.
- **Security Event Logger:** This component provides comprehensive audit trails. Utilizing Python's built-in logging module, it outputs structured JSON logs detailing authentication

successes, failures, file access attempts, and system errors, which is critical for forensic analysis and intrusion detection.

4. Technology Stack Justification

The selected technology stack prioritizes development efficiency without sacrificing security integrity.

- **Python & Flask:** Python was chosen for its vast ecosystem of peer-reviewed security libraries. Flask, as a micro-framework, provides granular control over HTTP headers and request handling without the opaque abstractions often found in heavier frameworks like Django. This allows for precise implementation of security headers and input sanitization.
- **JSON File-Based Storage:** By utilizing JSON files rather than a SQL database, the application entirely eliminates the attack surface for SQL Injection (SQLi), consistently one of the most critical vulnerabilities in modern web applications.
- **Bcrypt & Fernet:** Bcrypt is utilized for password hashing due to its adaptive work factor, which intentionally slows down hash computation to thwart brute-force and dictionary attacks. Fernet was selected for data-at-rest encryption because it guarantees that a message encrypted using it cannot be manipulated or read without the key, automatically implementing AES in CBC mode with a 128-bit key for encryption and HMAC using SHA256 for authentication.

B. Threat Model

1. Asset Identification

To effectively secure the system, the following critical assets must be protected:

- **Confidential Documents:** The raw, unencrypted files uploaded by users, containing sensitive information.
- **User Credentials:** Stored password hashes and user identification data.
- **Session Tokens:** Active tokens that grant authenticated access to the system.
- **Master Encryption Key (secret.key):** The symmetric key required to decrypt all stored documents.
- **Audit Logs:** Records of system activity, essential for non-repudiation and identifying malicious behavior.

2. Threat Enumeration

Based on the system's architecture and the OWASP Top 10, the system faces the following primary threats:

- **Path Traversal / Directory Climb:** An attacker manipulating file paths during upload or download to access or overwrite system files outside the designated web root.
- **Cross-Site Scripting (XSS):** Malicious scripts injected into document names or user inputs that execute in a victim's browser.

- **Broken Access Control:** A user manipulating requests to view or edit documents they do not own or have explicitly been granted access to.
- **Brute Force Attacks:** Automated, rapid attempts to guess user passwords and gain unauthorized entry.
- **Session Hijacking:** Interception of session cookies over insecure networks.

3. Vulnerability Assessment

The most severe vulnerabilities stem from improper input handling and poor access configurations. If file upload parameters are not strictly sanitized, the server is vulnerable to arbitrary code execution, where an attacker could upload and run a malicious Python or shell script. Furthermore, if the system relies solely on the UI to hide unauthorized actions, an attacker could bypass the frontend and directly query API endpoints to achieve vertical privilege escalation, highlighting the need for strict, server-side authorization checks on every sensitive route.

4. Attack Scenarios

- **Scenario A: The Malicious Upload (Path Traversal):** An attacker registers a standard user account and attempts to upload a file named ../../../etc/passwd. Without intervention, the operating system might resolve this path and overwrite critical system files. To mitigate this, the application utilizes secure_filename() to strip special characters and strictly verifies that the absolute path of the generated file resolves inside the intended DOCUMENTS_DIR.

- **Scenario B: Brute Force Credential Stuffing:** An attacker acquires a list of compromised passwords from a third-party data breach and deploys an automated script to test them against an administrative account on the application. The system's rate-limiting mechanism will restrict the attacker to 10 attempts per minute , and the account lockout protocol will freeze the targeted account for 15 minutes after 5 consecutive failures, rendering the automated attack mathematically unfeasible.

5. Risk Prioritization

Risks are prioritized based on likelihood and potential impact:

1. **Critical Risk:** Compromise of the secret.key. If this file is accessed via path traversal or server compromise, all data-at-rest encryption is instantly defeated.
2. **High Risk:** Broken Access Control. Failure to properly validate document ownership before serving a file could result in massive unauthorized data exposure.
3. **Medium Risk:** Brute Force Attacks. While severe, the implemented rate limiting and account lockouts significantly reduce the likelihood of a successful breach.
4. **Low Risk:** XSS. Given the system's heavy reliance on JSON APIs and backend processing rather than complex frontend rendering, the attack surface for XSS is minimized, further mitigated by strict HTML escaping on outputs.

C. Security Controls

1. User Authentication & Access Control

Control Description: The system enforces strict credential validation. Passwords must be at least 12 characters long and contain a mix of uppercase, lowercase, numerical, and special characters. Authentication attempts are heavily regulated: the system implements an IP-based rate limit of 10 requests per minute and a hard account lockout for 15 minutes after 5 consecutive failed attempts. Access is governed globally by a Role-Based Access Control (RBAC) matrix and locally via document-level permissions (Owner, Editor, Viewer).

Implementation Approach: Passwords are never stored in plaintext. They are hashed using the bcrypt library with a cost factor of 12, ensuring a mathematically expensive operation that deters brute-force cracking. Access control is implemented via a Python decorator pattern (@require_auth, @require_role). For document-level access, the has_doc_access() function dynamically evaluates the user's identity against the specific metadata arrays (viewers, editors) stored in documents_meta.json.

Testing Methodology: Authentication limits will be tested using automated fuzzing tools like OWASP ZAP to send rapid, successive login requests. This will verify that the HTTP 429 (Too Many Requests) and HTTP 403 (Forbidden - Account Locked) status codes are properly triggered.

Known Limitations: IP-based rate limiting cannot distinguish between multiple legitimate users sharing a single NAT gateway (e.g., a corporate office or public Wi-Fi), potentially locking out innocent users if one malicious actor shares the IP.

Mitigation Strategies: Future iterations should decouple rate limiting from IP addresses by implementing account-specific thresholding combined with CAPTCHA challenges to verify human interaction before applying a hard IP block.

2. Input Validation & Injection Prevention

Control Description: To protect against arbitrary code execution, system manipulation, and Cross-Site Scripting (XSS), all user inputs are treated as untrusted. Files are verified against a strict whitelist of allowed extensions (txt, pdf, png, jpg, jpeg, csv), and output rendered to the client is aggressively sanitized.

Implementation Approach: Path traversal is mitigated using Werkzeug's secure_filename() utility, which strips malicious path characters (like ../). Furthermore, the system resolves the absolute path of the generated file and verifies it begins with the exact DOCUMENTS_DIR base path, explicitly preventing directory climbing. To prevent XSS, Python's built-in html.escape() is used on any user-provided string (like filenames) before returning it in JSON responses.

Testing Methodology: Input vectors will be tested with known payload dictionaries (e.g., standard XSS vectors and path traversal strings like ../../etc/shadow) using Burp Suite. Uploads will be tested by modifying the file extension of a script payload to bypass basic checks.

Known Limitations: The mimetypes.guess_type() function relies on the file extension and operating system registry, meaning it can be spoofed by an attacker who renames a malicious executable to .pdf.

Mitigation Strategies: Implementing deep file inspection or integrating an antivirus daemon (like ClamAV) to scan the actual byte signatures of uploaded files would provide definitive malware protection over simple MIME-guessing.

3. Session Management

Control Description: The system uses a stateless, custom-built token architecture rather than relying exclusively on default framework sessions. Sessions are automatically invalidated after 30 minutes of inactivity.

Implementation Approach: Upon login, the secrets.token_urlsafe(32) function generates a cryptographically secure token with high entropy. The application stores this token mapped to the user's IP address, User-Agent, and activity timestamps. Tokens are delivered to the client via HTTP cookies flagged with httponly=True (preventing JavaScript access) and secure=True (forcing transmission over encrypted channels only).

Testing Methodology: Session timeouts will be tested manually by authenticating, waiting 31 minutes, and attempting to access a protected resource. Session fixation will be tested by attempting to pass a pre-defined token in the request header prior to authentication.

Known Limitations: The current system does not actively monitor for concurrent sessions. A user could theoretically be logged in from multiple devices simultaneously, increasing the attack surface if one device is compromised.

Mitigation Strategies: Implementing a mechanism to track active tokens per user and automatically invalidating older tokens when a new login occurs would enforce a single-active-session policy.

4. Encryption & Security Headers

Control Description: Data is protected comprehensively both in transit and at rest. Additionally, the browser's execution environment is hardened using a suite of HTTP security headers.

Implementation Approach: Transport encryption is enforced by loading an ssl_context in the Flask server. Data-at-rest is encrypted using the cryptography.fernet library, an implementation of symmetric AES-128 in CBC mode. For headers, an @app.after_request interceptor injects Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security into every outbound response.

Testing Methodology: Headers will be verified using the SecurityHeaders.com testing suite. Transport encryption will be audited by using a packet sniffer (like Wireshark) to ensure no plaintext traffic is visible.

Known Limitations: The application generates self-signed certificates for local development, which modern browsers will flag as insecure, requiring manual user exceptions.

Mitigation Strategies: For a production deployment, the application would utilize an automated Certificate Authority (such as Let's Encrypt) to provision universally trusted TLS certificates.

D. Data Protection

1. Data Classification

To apply appropriate security controls, system data is classified into three distinct tiers:

- **Restricted:** The master encryption key (secret.key) and password hashes. Unauthorized disclosure of this data compromises the entire system.
- **Confidential:** User-uploaded documents, session tokens, and the documents_meta.json file detailing user access relationships.
- **Internal:** The security.log files. While not containing raw user data, they hold sensitive operational telemetry (IP addresses, user agents) that could aid an attacker in reconnaissance.

2. Encryption Methods

The system employs a dual-layer cryptographic approach.

- **In Transit:** All data traveling between the client browser and the server is encrypted using TLS/HTTPS protocols. This prevents Man-in-the-Middle (MitM) attacks from sniffing sensitive uploads or session cookies.
- **At Rest:** Before any document is written to the physical storage disk, the raw byte stream is passed through the Fernet symmetric encryption cipher. This ensures that even if an attacker bypasses the web application and gains direct filesystem access to the server, the documents remain mathematically undecipherable.

3. Key Management

The integrity of the Data-at-Rest encryption relies entirely on the secrecy of the master key.

- **Generation:** The key is generated automatically by the Fernet.generate_key() method during the application's initial startup if one does not already exist.
- **Storage:** The key is stored locally in the data/secret.key file.
- **Protection:** Because the key is stored on the same server as the encrypted data, the primary defense mechanism is operating-system-level access control. The file permissions must be strictly scoped so that only the user executing the Flask application daemon has read access.

4. Secure Deletion Procedures

Standard file deletion on modern operating systems merely removes the pointer to the data, leaving the raw bytes recoverable on the disk until they are overwritten. The Secure Document Sharing System approaches this through the concept of **Cryptographic Erasure**.

Because all uploaded documents are encrypted with the master key, deleting the encrypted binary blob from the DOCUMENTS_DIR is sufficient. Even if the raw encrypted bytes are recovered from the disk by a forensic utility, they are useless without the key. If a total system purge is required, securely destroying the secret.key instantly and permanently renders all stored documents globally unrecoverable, fulfilling secure deletion requirements without requiring multi-pass disk wiping.